

QUIC Short Header packets (also called **1-RTT packets**) transport application data.

Specifically, **STREAM frames** carried within protected 1-RTT packets are the primary mechanism for transporting data in QUIC. Here's a quick breakdown of the packet types:

Long Header Packets (used during handshake):

- **Initial** – carries CRYPTO frames for the TLS handshake
- **Handshake** – carries more CRYPTO frames to complete TLS
- **0-RTT** – carries early application data (before handshake completes)
- **Retry** – used for address validation

Short Header Packets (used after handshake):

- **1-RTT** – the main data-carrying packet type; transports STREAM frames with actual application data

1-RTT packets (short header) are the primary packet type for transporting data, though **0-RTT packets** can also carry application data early in the connection setup.

HEADER BYTE (1 byte) Bit 7 (MSB): Header Form = 0 (Short Header) Bit 6: Fixed Bit = 1 Bit 5: Spin Bit (latency measurement) Bit 4: Reserved (must be 0) Bit 3: Key Phase Bit Bits 1-2: Reserved (must be 0) Bits 0-1: Packet Number Length (2-bit field)
DESTINATION CONNECTION ID (variable length, 0-20 bytes)
PACKET NUMBER (1, 2, 3, or 4 bytes)
PAYLOAD (one or more QUIC frames, e.g. STREAM) encrypted with AEAD (AES-GCM or ChaCha20-Poly1305)
AEAD Authentication Tag (16 bytes)

Field Descriptions

<i>Field</i>	<i>Size</i>	<i>Description</i>
Header Byte	1 byte	Flags and packet number length
Destination Connection ID	0–20 bytes	Identifies the connection on the receiver side
Packet Number	1–4 bytes	Monotonically increasing, used for ordering & ACKs
Payload	Variable	Encrypted QUIC frames (STREAM, ACK, etc.)
AEAD Tag	16 bytes	Integrity/authentication tag from encryption

Payload:

General Frame Structure

Frame Type (variable length int)	Frame Data (type-specific)
--	-------------------------------

Frame Type is encoded as a Variable-Length Integer (VarInt)

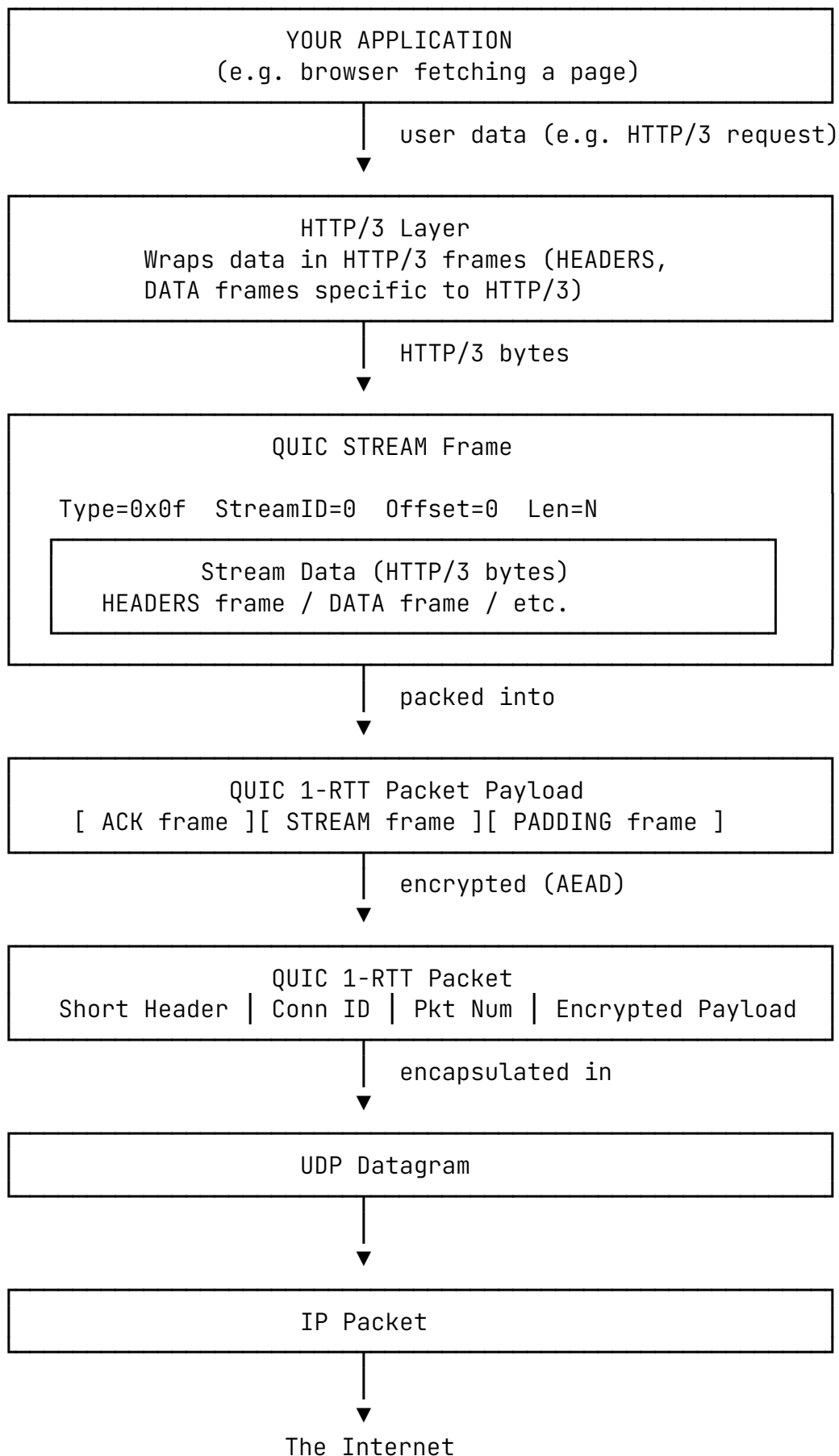
VarInt Encoding (RFC 9000)	
2-bit prefix determines total length:	
00xxxxxx	= 1 byte (0 to 63)
01xxxxxx xxxxxxxx	= 2 bytes (0 to 16383)
10xxxxxx xxxxxxxx ...	= 4 bytes (0 to 2 ³⁰ -1)
11xxxxxx xxxxxxxx ...	= 8 bytes (0 to 2 ⁶² -1)

Complete Frame Type Table

TYPE	NAME	PURPOSE
0X00	PADDING	No-op, used for amplification protection
0X01	PING	Elicit an ACK, keep-alive
0X02–0X03	ACK	Acknowledge received packets
0X04	RESET_STREAM	Abruptly terminate a stream
0X05	STOP_SENDING	Request peer stops sending on a stream

0X06	CRYPTO	TLS handshake data
0X07	NEW_TOKEN	Provide token for future connections
0X08– 0X0F	STREAM	Application data
0X10	MAX_DATA	Connection-level flow control
0X11	MAX_STREAM_DATA	Stream-level flow control
0X12– 0X13	MAX_STREAMS	Increase max stream limit
0X14	DATA_BLOCKED	Blocked at connection level
0X15	STREAM_DATA_BLOCKED	Blocked at stream level
0X16– 0X17	STREAMS_BLOCKED	Cannot open more streams
0X18	NEW_CONNECTION_ID	Provide alternative connection IDs
0X19	RETIRE_CONNECTION_ID	Retire a connection ID
0X1A	PATH_CHALLENGE	Path validation probe
0X1B	PATH_RESPONSE	Reply to path challenge
0X1C– 0X1D	CONNECTION_CLOSE	Terminate the connection
0X1E	HANDSHAKE_DONE	Server signals handshake complete

How data travels on the internet actually?

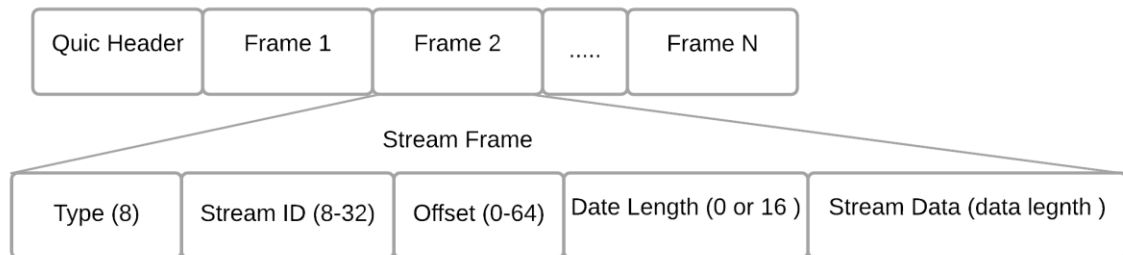


How to identify different types of data from stream frame:

STREAM Frame – Raw Structure

[Type][Stream ID][Offset][Length][Data...]

The "Data" field is just BYTES – QUIC doesn't care whether it's HTML, a JPEG, or a game position update



Layer stack:

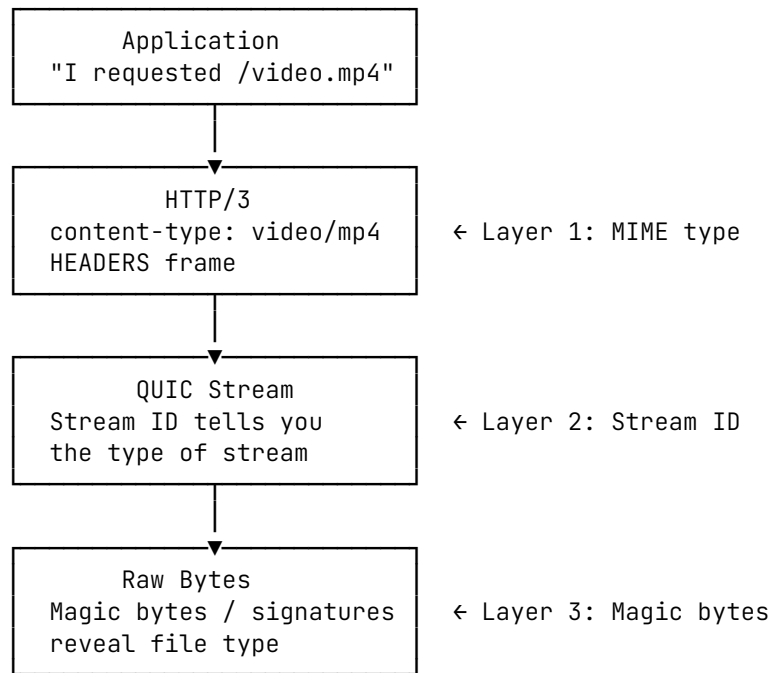
Application (Browser)	
HTTP/3	← HEADERS + DATA frames
QPACK (compression)	← compresses HTTP headers
QUIC STREAM Frame	← carries HTTP/3 bytes
QUIC 1-RTT Packet	
UDP / IP	

Few things we can infer :

	Stream ID	Offset	FIN	Data inside
HTML page	0	0→N	last frame	HTTP/3 HEADERS+DATA
CSS file	4	0→N	last frame	HTTP/3 HEADERS+DATA
JPEG image	8	0→N	last frame	HTTP/3 DATA (binary)
JS file	12	0→N	last frame	HTTP/3 HEADERS+DATA
Game pos	0	0	1 (single)	raw binary struct
Chat msg	4,8,12...	0	1 (single)	raw binary struct
Video chunk	4	0→huge	last frame	HTTP/3 DATA (binary)

The Key Difference is Chunking

Binary data is typically **large**, so it gets split across **multiple STREAM frames**, each with an incrementing offset.



QUIC packets are data-agnostic — the payload is just encrypted bytes, so identification happens at higher layers. The most reliable method is the **HTTP/3 content-type header** (Layer 1), which explicitly declares the MIME type (e.g. image/jpeg, video/mp4) in the HEADERS frame that precedes the DATA frame on every stream. If headers aren't available, **Stream ID patterns** (Layer 2) can hint at the stream's purpose — for example, unidirectional streams carry control or QPACK data, while client-initiated bidirectional streams carry actual request/response data — though this tells you the stream's role, not the data type itself. As a last resort, **magic bytes** (Layer 3) at the start of the raw payload can identify the file type by their fixed byte signatures, such as FF D8 FF for JPEG or 89 50 4E 47 for PNG. In practice, you always check content-type first, fall back to magic bytes if needed, and treat Stream ID as structural context rather than a data type indicator.

Byte Signatures	
Data Type	First Bytes (hex)
JPEG	FF D8 FF
PNG	89 50 4E 47 0D 0A 1A 0A
GIF	47 49 46 38 ("GIF8")
WebP	52 49 46 46 ... 57 45 42 50 ("RIFF..WEBP")
MP4	... 66 74 79 70 ("ftyp" at offset 4)
WebM	1A 45 DF A3 (EBML header)
MP3	FF FB or 49 44 33 ("ID3")
PDF	25 50 44 46 ("%PDF")
ZIP	50 4B 03 04 ("PK")
HTML	3C 21 44 4F ("<!DOCTYPE") or 3C 68 74 ("<html>")
JSON	7B or 5B ('{' or '[')
Plain text	printable ASCII bytes

Observe QUIC flow



Is it Long Header (Initial/Handshake)?

YES → Handshake phase, no user data yet

NO → Short Header (1-RTT), data flowing



Packet size distribution?

- └ Mostly ~1200B → Bulk data (file/video)
- └ Mostly ~50-300B → Interactive (chat/game/VoIP)
- └ Mixed → Web browsing



Timing pattern?

- └ Continuous flood → File download/upload
- └ Burst-pause-burst → ABR video streaming
- └ Fixed ~20ms interval → VoIP/audio
- └ Irregular small bursts → Gaming/chat



Directionality?

- └ Asymmetric (S→C heavy) → Download/streaming
- └ Asymmetric (C→S heavy) → Upload
- └ Symmetric → Video call/gaming



Flow duration?

- └ < 5s → Web request, API, small file
- └ 5-30s → Medium file, short video
- └ > 30s → Live stream, large download, call



If endpoint/proxy – decrypt and check:

- └ content-type header → exact media type
- └ :path pattern → /video/, /api/, /download/
- └ Stream count + size → page load vs single file